

MANUAL TECNICO

Estructura General del Programa:

El sistema fue hecho en visual estudio code con el lenguaje de C++ utilizando programación orientada a objetos(POO). El proyecto está desarrollado de una forma modular, separando declaraciones de clases (.h) e implementaciones (.Cpp).

Diagrama de Componentes UML

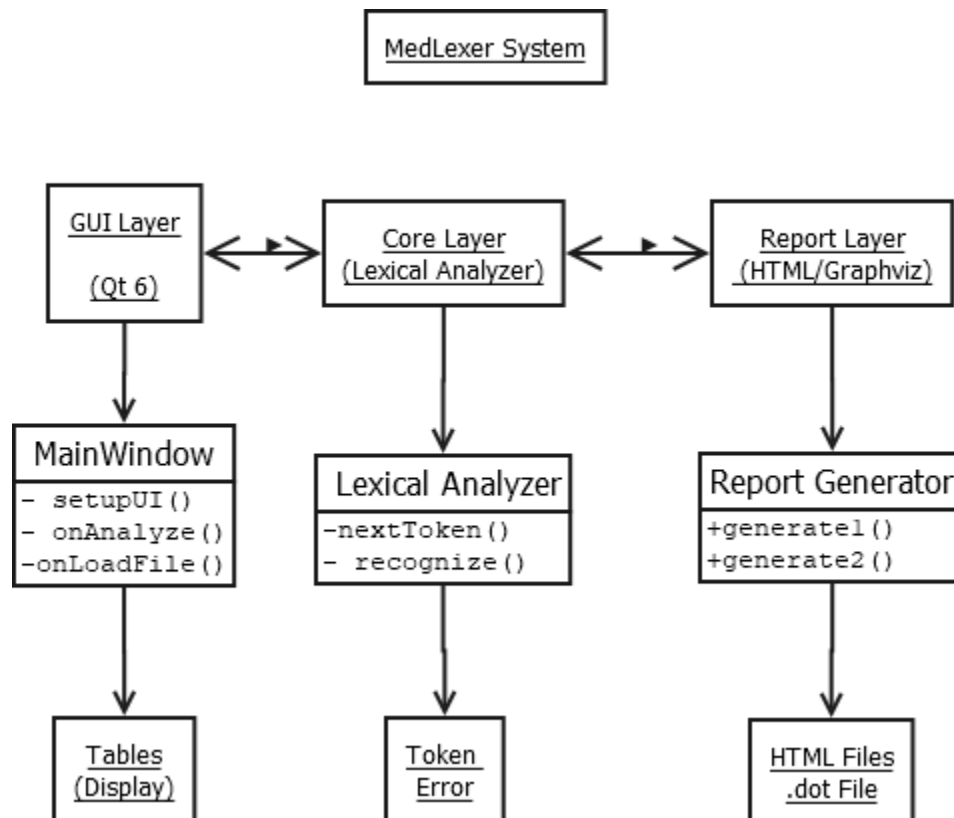
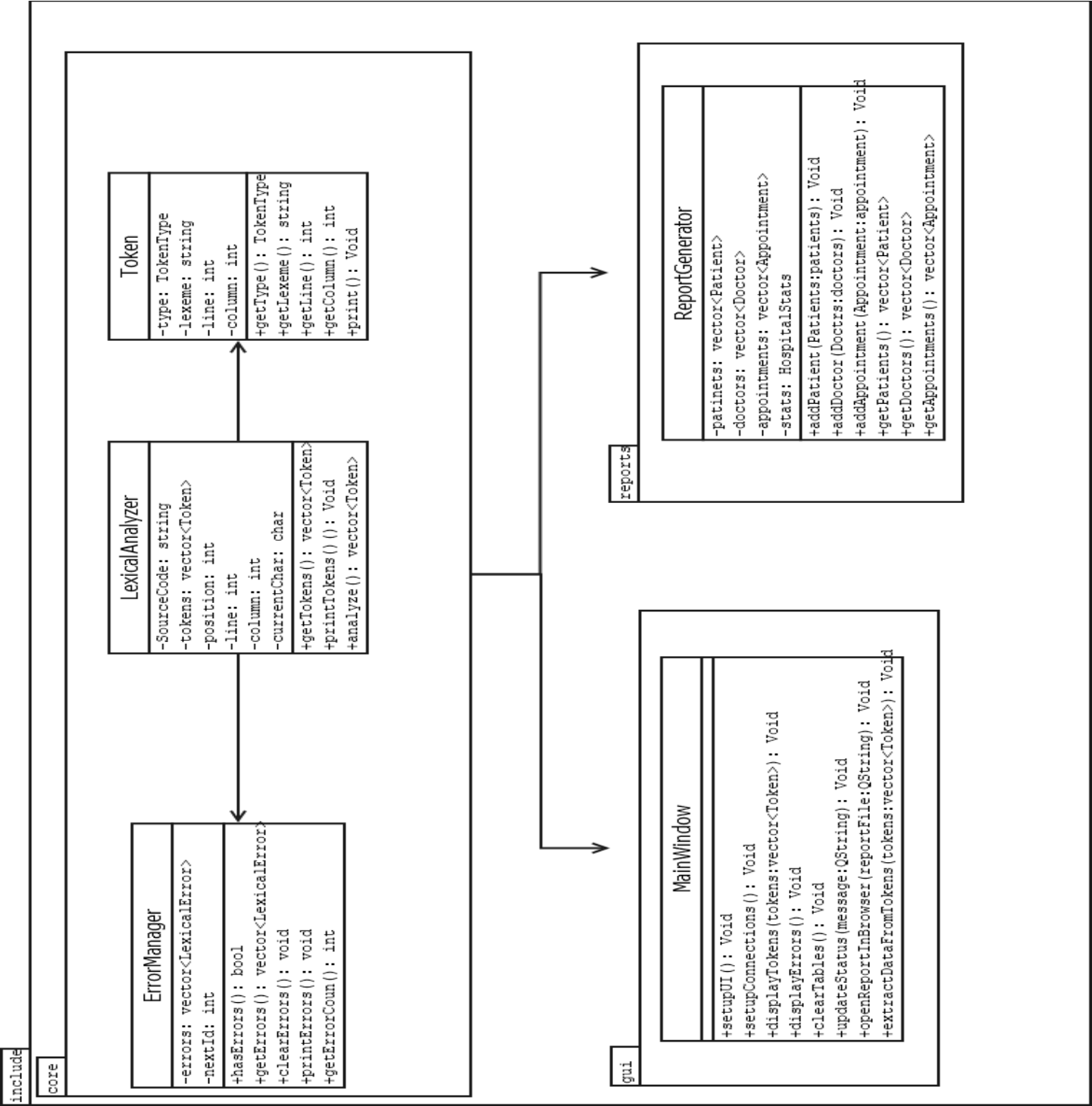


Diagrama de Clases Completo



Diagramas del AFD

Básicas:

$L = [a-zA-Z]$
$N = [0-9]$
$S = \{ \} ; , \backslash [\] : " " "$
$W = [\backslash t \backslash n \backslash r]$

Palabras Reservadas:

HOSPITAL	= "HOSPITAL"
PACIENTES	= "PACIENTES"
MEDICOS	= "MEDICOS"
CITAS	= "CITAS"
DIAGNOSTICOS	= "DIAGNOSTICOS"
paciente	= "paciente"
medico	= "medico"
cita	= "cita"
diagnostico	= "diagnostico"
con	= "con"

1. Generalización

ER:

$((L)^* | N^* | (L) | S^*) \#$

Diagrama:

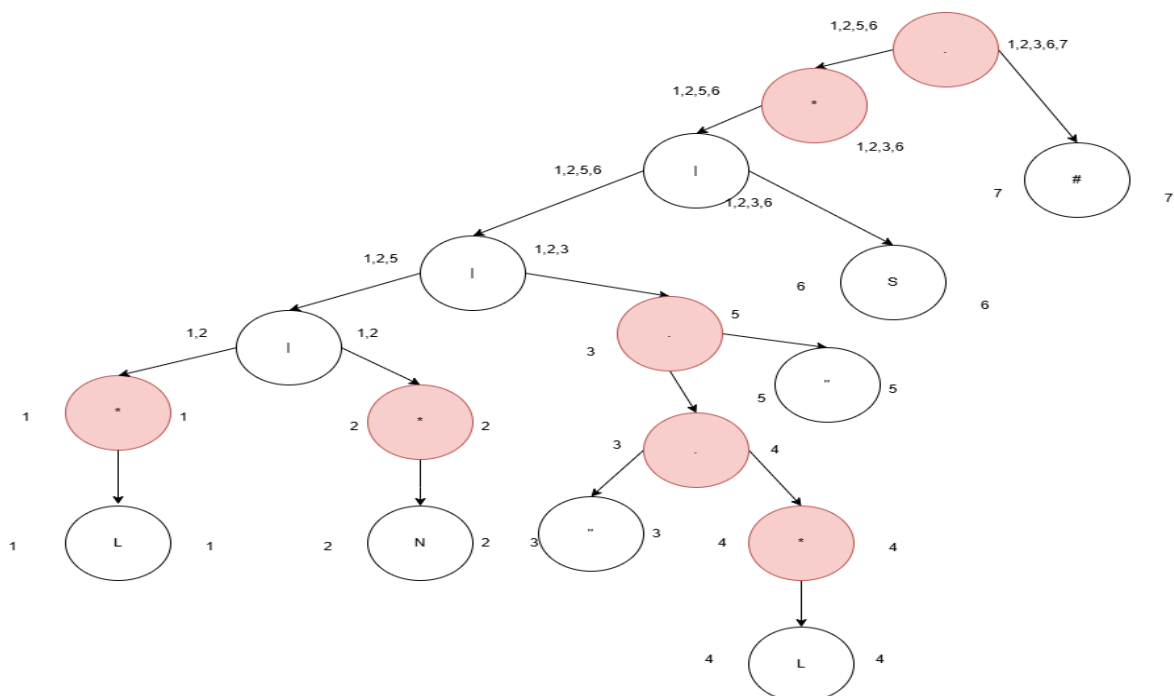
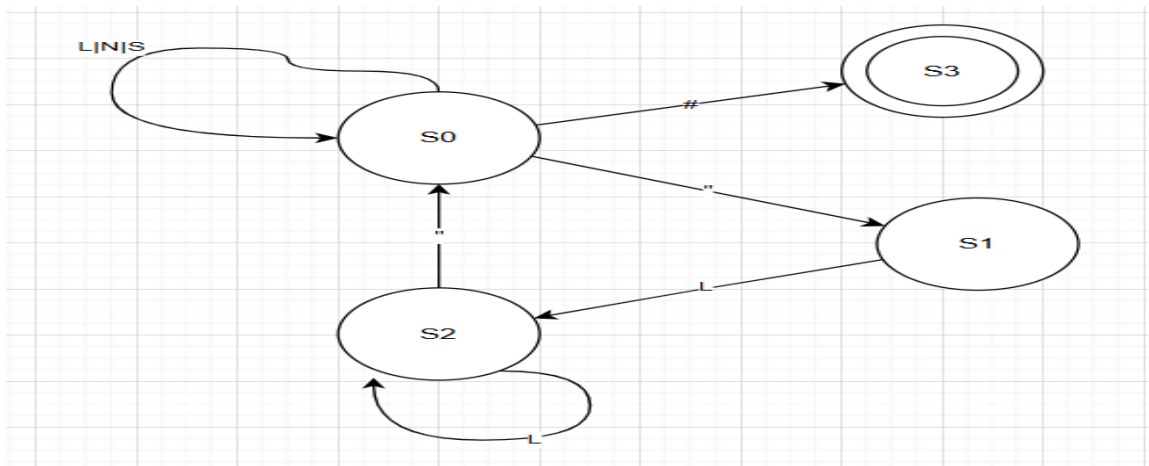


Tabla:

HOJA	TERMINAL	SIGUIENTES
1	L	1,2,3,6
2	N	1,2,3,6
3	"	4
4	L	4,5
5	"	1,2,3,6
6	S	1,2,3,6
7	#	-

Transiciones del autómata:

$S0 = \{1,2,3,6\}$	$L = S0 = \{1,2,3,6\}$
$S0 = \{1,2,3,6\}$	$N = S0 = \{1,2,3,6\}$
$S0 = \{1,2,3,6\}$	$" = S0 = \{1,2,3,6\}$
$S0 = \{1,2,3,6\}$	$S = S0 = \{1,2,3,6\}$
$S1 = \{4\}$	$L = S2\{4,5\}$
$S2\{4,5\}$	$" = S0 = \{1,2,3,6\}$
$S3 = 7$	-



2. Fecha (Formato: AAAA-MM-DD):

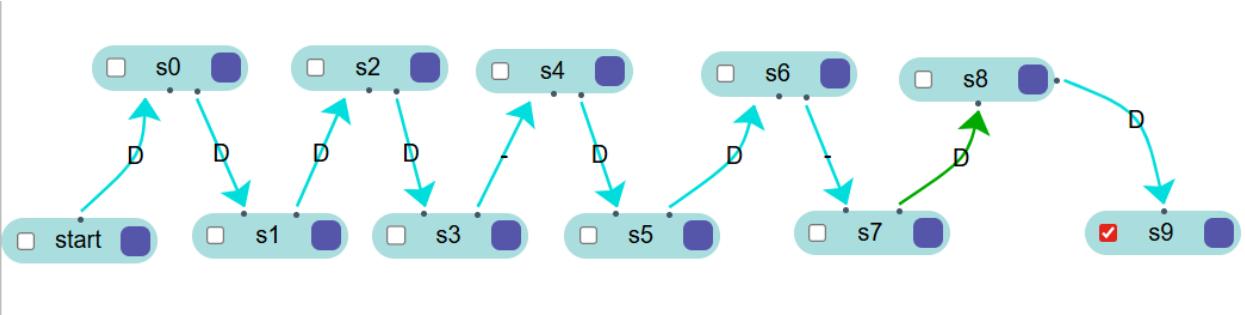
ER:

$([0-9][0-9][0-9][0-9]) - ([0-9][0-9]) - ([0-9][0-9])$

D: Dígitos, X= Trampa

$\Sigma = \{D, -\}$

	D	-
->Star	S0	X
S0	S1	X
S1	S2	X
S1	S3	X
S3	X	S4
S4	S5	X
S5	S6	X
S6	X	S7
S7	S8	X
S8	S9	X
*S9	X	X



3. Código de Identificación con Guion (Formato: PAC-001, MED-023)

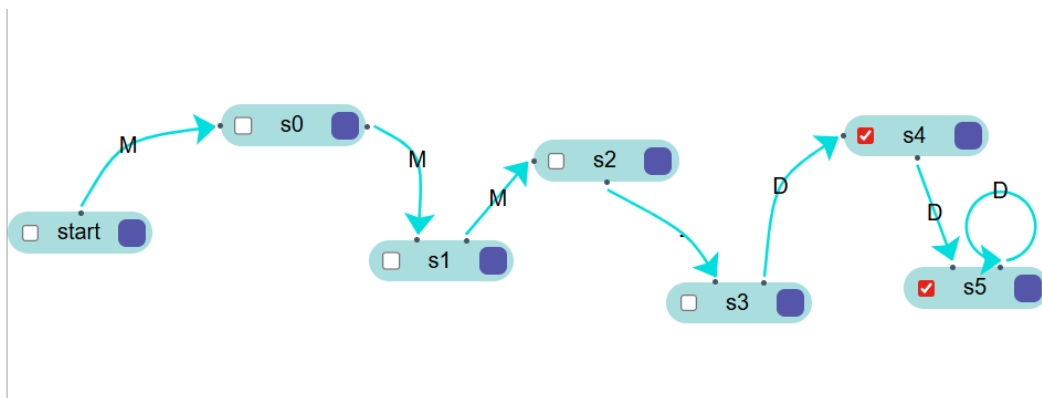
ER:

([A-Z]) ([A-Z]) ([A-Z]) – ([0-9]) +

M: Mayúsculas, D: Dígitos, X= Trampa

$\Sigma = \{M, D, -\}$

	M	D	-
->Star	S0	X	X
S0	S1	X	X
S1	S2	X	X
S2	X	X	S3
S3	X	S4	X
*S4	X	S5	X
*S5	X	S5	X

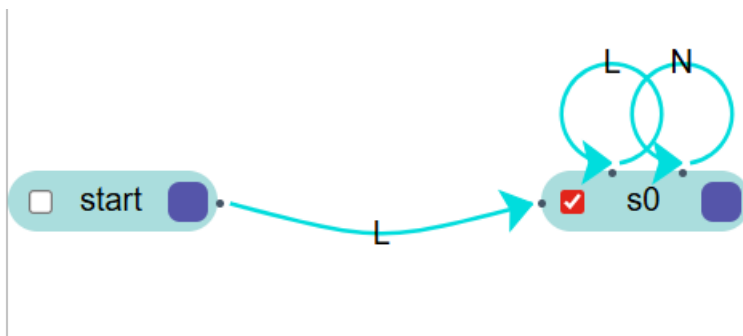


4. IDENTIFICADORES

ER:

$$ID = L(L|N)^*$$

q0	L -> q1
q1	(L N)--> q1
q1	otro--> ACEPTA



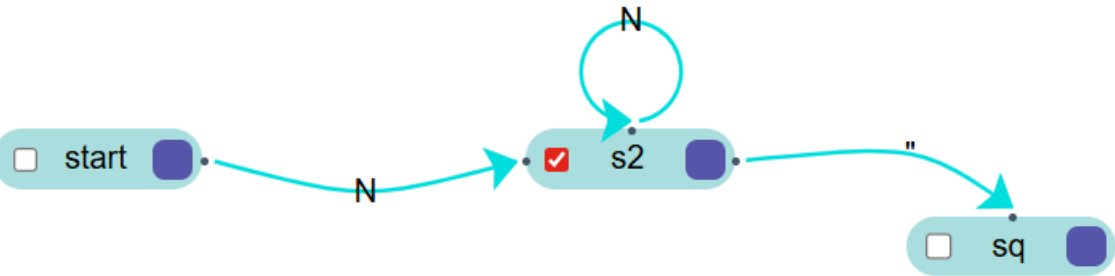
5. ENTERO

ER:

$$N^+$$

qE = estado de error

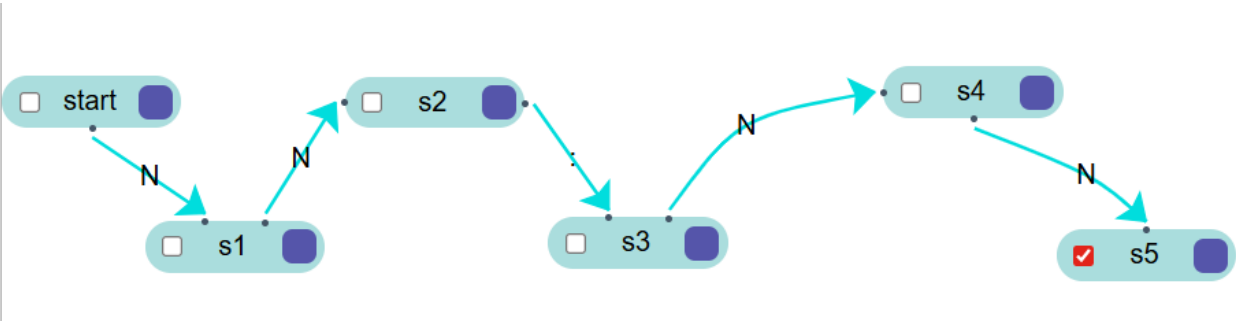
q0	N--> q2
q2	N--> q2
q2	"---> qE
q2	"---> qE



6. Hora

ER: N N ":" N N

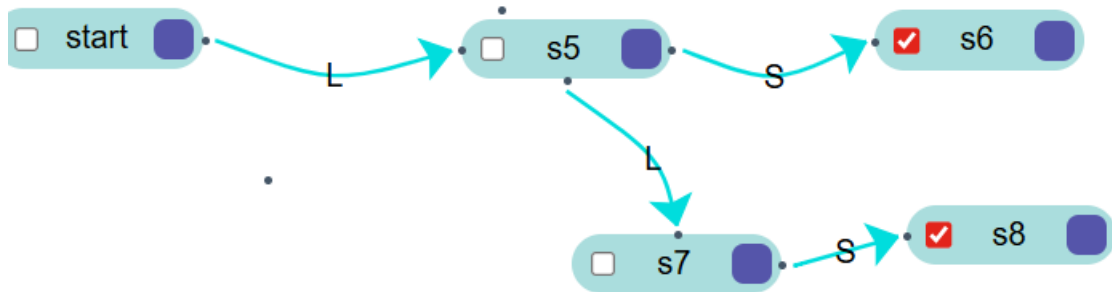
q0	N--> q1
q1	N--> q2
q2	: --> q3
q3	N--> q4
q4	N--> q5



7. TIPO DE SANGRE

ER: "A+" | "A-" | "B+" | "B-" | "O+" | "O-" | "AB+" | "AB-"

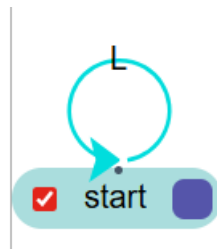
q0	L--> q1
q1	L--> q2
q1	S --> q1



8. ESPECIALIDADES

ER: ESPECIALIDAD = "CARDIOLOGIA" | "NEUROLOGIA" | "PEDIATRIA" | "CIRUGIA" | "MEDICINA_GENERAL" | "ONCOLOGIA"

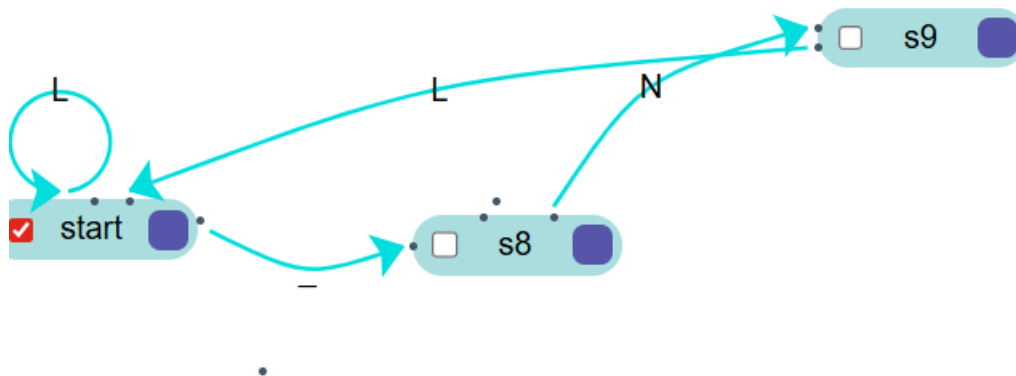
q0	L--> q0
----	---------



9. DOSIS

ER: DOSIS = "DIARIA" | "CADA_8_HORAS" | "CADA_12_HORAS" | "SEMANAL"

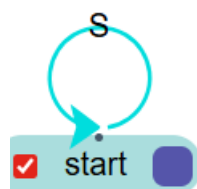
q0	L--> q0
q0	"--> q1
q1	N--> q2
q2	L--> q0



10. SÍMBOLOS

ER: LLAVE_ABRE = "{"
 LLAVE_CIERRA = "}"
 CORCHETE_ABRE = "["
 CORCHETE_CIERRA = "]"
 DOS_PUNTOS = ":"
 COMA = ","
 PUNTO_COMA = ";"

q0	S--> q0
----	---------



Algoritmos

ALGORITMO nextToken()

ENTRADA: carácter actual currentChar

SALIDA: Token

1. SI es letra (a-z, A-Z) ENTONCES
2. llamar recognizelIdentifier()
3. SINO SI es dígito (0-9) ENTONCES
4. llamar recognizeNumberOrDateTime()
5. SINO SI currentChar = "" ENTONCES
6. llamar recognizeString()
7. SINO
8. reconocer delimitadores individuales
9. SI es delimitador válido ENTONCES
10. avanzar y retornar token
11. SINO
12. reportar error: carácter ilegal
13. FIN SI

ALGORITMO recognizelIdentifier()

ENTRADA: posición actual

SALIDA: Token

1. lexeme = ""
2. MIENTRAS no EOF Y (letra O dígito O '_') HACER
3. lexeme += currentChar
4. advance()
5. FIN MIENTRAS
6. SI lexeme es palabra reservada ENTONCES
7. retornar Token(RESERVED, lexeme)
8. SINO SI lexeme es atributo (edad, tipo_sangre, etc.) ENTONCES
9. retornar Token(ATRIBUTO, lexeme)
10. SINO SI lexeme es especialidad médica ENTONCES
11. retornar Token(ESPECIALIDAD, lexeme)
12. SINO SI lexeme es tipo de dosis ENTONCES
13. retornar Token(DOSIS, lexeme)
14. SINO SI lexeme es tipo de sangre ENTONCES

15. retornar Token(BLOOD_TYPE, lexeme)
16. SINO
17. errorManager->addError(lexeme, "Token inválido")
18. retornar Token(ERROR, lexeme)
19. FIN SI

ALGORITMO isValidDate(dateStr)

ENTRADA: string con formato AAAA-MM-DD

SALIDA: booleano

1. SI length(dateStr) != 10 ENTONCES retornar FALSO
2. SI dateStr[4] != '-' O dateStr[7] != '-' ENTONCES retornar FALSO
3. año = stoi(dateStr[0:4])
4. mes = stoi(dateStr[5:7])
5. día = stoi(dateStr[8:10])
6. SI mes < 1 O mes > 12 ENTONCES retornar FALSO
7. díasPorMes = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
8. SI mes == 2 ENTONCES
9. esBisiesto = (año % 4 == 0 AND año % 100 != 0) OR (año % 400 == 0)
10. SI esBisiesto ENTONCES díasPorMes[1] = 29
11. FIN SI
12. retornar día >= 1 AND día <= díasPorMes[mes-1]

ALGORITMO manejoErrores()

ENTRADA: error detectado

SALIDA: continuación del análisis

1. Registrar error con:
2. número incremental
3. lexema que causó el error
4. tipo de error
5. descripción
6. línea y columna actual
7. gravedad (ERROR o CRÍTICO)
8. SI gravedad == ERROR ENTONCES
9. continuar análisis normalmente
10. SINO SI gravedad == CRÍTICO ENTONCES
11. saltar hasta el siguiente delimitador seguro

- 12. continuar análisis
- 13. FIN SI
- 14. Al finalizar, mostrar todos los errores acumulados

Justificación de Decisiones de Diseño

Criterio	Justificación
Rendimiento	C++ ofrece alta eficiencia para procesamiento de texto
Control	Permite implementación manual del AFD sin abstracciones
STL	std::string, std::vector, std::map útiles para el proyecto
Qt Integration	Compatibilidad nativa con Qt para la GUI

Implementación Manual del AFD

Decisión: No usar std::regex ni herramientas automáticas (Flex/Bison)

Justificación:

- Mayor control sobre el proceso de tokenización
- Mejor comprensión del funcionamiento de autómatas
- Seguimiento preciso de línea y columna
- Recuperación de errores personalizada

Estructura de Clases

Decisión: Separación en capas (Core, Reports, GUI)

Justificación:

- Mantenibilidad: Cada clase tiene una responsabilidad única
- Reusabilidad: El analizador léxico puede usarse sin GUI
- Testabilidad: Las capas pueden probarse independientemente

Manejador de Errores (ErrorManager)

Decisión: Acumular errores y continuar el análisis

Justificación:

- Permite detectar múltiples errores en una sola pasada
- Mejor experiencia de usuario al ver todos los errores
- El requisito del proyecto exige recuperación de errores

Generación de Reportes HTML

Decisión: HTML con CSS embebido

Justificación:

- Portabilidad: Los reportes se abren en cualquier navegador
- Sin dependencias externas para visualizar
- Formato profesional y legible

Interfaz Gráfica con Qt

Decisión: Qt 6 sobre wxWidgets

Justificación:

- Documentación extensa y comunidad activa
- Mejor integración con CMake
- Herramientas de diseño (Qt Designer)
- Soporte multiplataforma

Tablas de Tokens y Errores

Decisión: QTableWidget para visualización

Justificación:

- Fácil de implementar
- Ordenamiento y filtrado automático

- Alternancia de colores para mejor legibilidad

Validación de Tipos de Datos

Decisión: Validación manual sin regex

Justificación:

- Cumple con la restricción del proyecto (no std::regex)
- Mayor precisión en mensajes de error
- Control total sobre el formato esperado